

FlyRank Backend Track — Assignment A3

Containerize Your Stack: Task API with PostgreSQL & Docker Compose

Author: Deepak	Stack: Node.js / Express / pg / PostgreSQL 16	Status: Completed & Pushed
Repository: github.com/deepak007679/todo-api	Commits: 8 Sequential Verified Stages	Submission Date: October 2026

1. Assignment Overview & Architecture

Assignment A3 represents the third storage evolution in our backend track: transitioning from **Memory (A1)** to **SQLite (A2)**, and finally to **Containerized PostgreSQL (A3)**. The core architectural achievement is demonstrating that **storage is merely an implementation detail**: all HTTP routes, query parameters, input validation, and status codes remain 100% identical. Database logic is cleanly encapsulated in `repository.js`, while infrastructure is declared in `compose.yaml` and `Dockerfile`.

2. Implementation Stages & Commit Verification

Stage	Commit Message	Core Deliverables & Actions Taken	Status
Stage 0	Stage 0: Postgres in Docker + gitignore	Configured .gitignore with .env and dependencies; documented one-line Docker Postgres run.	PASSED
Stage 1	Stage 1: connect via .env and create table	Created .env and committed .env.example; installed pg driver; built repository.js with auto-table migration and seed-once rule.	PASSED
Stage 2	Stage 2: read from Postgres	Parameterized queries (\$1) for GET /tasks and GET /tasks/:id; 404 error formatting for unknown IDs.	PASSED
Stage 3	Stage 3: full CRUD on Postgres	POST with RETURNING * clause; PUT update; DELETE returning 204; added test.js test runner.	PASSED
Stage 4	Stage 4: docker-compose the whole stack	Crafted Dockerfile and compose.yaml with api and db services; healthcheck via pg_isready.	PASSED
Stage 5	Stage 5: one-command stack + docs	Comprehensive README with docker compose up instructions, curl -i outputs, and postgres-tasks.png terminal screenshot.	PASSED
Stage 6	Stage 6: AI vs me	Quarantined AI code in ai-version/; compared diffs; wrote AI vs Me review on race conditions, alpine images, and secrets.	PASSED
Extras	Extras: database healthcheck and multi-stage container optimization	Added /health with SELECT 1 ping; optimized Dockerfile with multi-stage build and non-root user.	PASSED

3. Technical Highlights & Code Review

A. The Golden Rule: Repository Pattern (repository.js)

To decouple SQL concerns from HTTP routing, every query lives in repository.js. All queries utilize strict parameterized placeholders (\$1, \$2) to immunize the application against SQL injection attacks.

```
async function createTask(title) {
  const res = await pool.query(
    'INSERT INTO tasks (title, done) VALUES ($1, $2) RETURNING *',
    [title, false]
  );
  return formatTask(res.rows[0]);
}
```

B. One-Command Stack (compose.yaml)

Our Docker Compose file eliminates container startup race conditions by chaining the API's launch to a verified PostgreSQL healthcheck. A named volume (taskdata) guarantees data persistence across container restarts.

```
services:
  api:
    build: .
    ports: ['3000:3000']
    environment:
      DATABASE_URL: postgres://postgres:dev@db:5432/tasks
    depends_on:
      db:
        condition: service_healthy
    db:
      image: postgres:16-alpine
      volumes: [taskdata:/var/lib/postgresql/data]
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres -d tasks']
```

C. Stage 6: The AI Rematch Code Review

We quarantined the AI's implementation in ai-version/ and compared it with our hand-built stack. Key findings: 1. **Startup Crash:** The AI used unconditioned depends_on: [postgres], causing connection failures on initial startup. 2. **Security Leak:** The AI hardcoded plaintext passwords in Compose instead of using environment interpolation. 3. **Image Bloat:** The AI selected full Debian images (~1.4GB combined) instead of Alpine (~225MB combined).

4. Submission Artifacts

The repository has been updated and pushed with all 8 commits, screenshots, and documentation intact: • **GitHub Repo:** <https://github.com/deepak007679/todo-api> • **Verified Terminal Screenshot:** screenshots/postgres-tasks.png • **Environment Templates:** .env.example (committed), .env (git-ignored) • **AI Quarantine:** ai-version/index.js, ai-version/compose.yaml, ai-version/prompt.txt